

1. What AGENTS.md Is For

AGENTS.md is durable project guidance that travels with a repository. It can tell Codex how the project is built, tested, reviewed, and organized before a task begins. It is not a replacement for README documentation, a full architecture history, or the user's current request.

High-value contents are boring in the best sense: exact build commands, exact test commands, generated-file rules, repository-specific naming, invariants that are easy to violate, and review expectations that are not otherwise machine-enforced.

2. Information Geometry

A root AGENTS.md should contain rules with broad scope. Nested files should add or override only what is special to the subtree. This mirrors the codebase itself: shared conventions at the trunk, local exceptions close to the branch that needs them.

Bad geometry produces either repetition or surprise. If three packages each copy the same twenty lines, they will drift. If a root file contains five pages of instructions for one package, every unrelated task pays the attention cost.

3. Commands Before Commentary

Compare:

Weak: "Make sure the code is high quality and thoroughly tested."

Strong: "For changes under packages/api, run ``pnpm test --filter api`` and ``pnpm lint --filter api``. For schema changes, also run ``pnpm db:check``."

The second version converts a virtue word into an executable obligation. It also creates evidence the final report can cite.

4. What Not to Put There

Do not store a one-off deadline, temporary branch name, current bug hypothesis, or personal request that applies only today. Do not paste long external manuals. Do not encode every stylistic preference in prose when a formatter can decide it. Do not write an instruction whose trigger is unknowable.

The cost of bloated guidance is not merely tokens. It increases the number of rules that can conflict, masks the rules that matter, and makes maintenance harder.

5. A Compact Pattern

A restrained repository file often needs only:

Project map: where important things live.

Commands: build, targeted tests, full tests, lint/type checks.

Conventions: only project-specific rules not enforced elsewhere.

Verification: what must be demonstrated before saying a task is complete.

Boundaries: generated files, migrations, production actions, credentials.

Nested guidance: pointers or a note that subtrees carry additional rules.

6. The Audit Exercise

Take each sentence in an existing AGENTS.md and label it one of five ways: keep; enforce in tooling; move to a skill; move closer to the relevant subtree; delete. Then ask Astra to perform a representative task before and after the audit. The goal is not to shorten the file for aesthetics. The goal is to increase the ratio of useful behavioral constraints to inert prose.

7. AGENTS.md versus Skill

A durable rule says what should normally be true. A skill describes how to perform a repeatable multi-step job.

"Use pnpm" is guidance. "To cut a release, update these files, run these checks, create this artifact, and stop before publishing" is a workflow. Conflating them turns the always-loaded instruction surface into a procedural encyclopedia.

References: OpenAI, Custom instructions with AGENTS.md; OpenAI, Codex customization; openai/codex repository.

Independent Open Course

<https://github.com/dicnunz/codex-astra-course>

CODEX.00 | Codex with GPT-6 Astra

Handout 6: AGENTS.md and Durable Rules

Fall 2026

License and source

Structural and pedagogical adaptation of MIT OpenCourseWare 24.00 Problems of Philosophy, Fall 2019, Prof. Miriam Schoenfield.

Original course and this adaptation are licensed CC BY-NC-SA 4.0.

This course is independently produced and is not affiliated with or endorsed by MIT or OpenAI.

Linked external readings and referenced product documentation retain their own copyrights and terms.

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

<https://ocw.mit.edu/courses/24-00-problems-of-philosophy-fall-2019/>